

Módulo 8 – Capítulo 10 – Sección 01

Cuándo usar local y cuándo usar nube: privacidad, latencia, costo y capacidad

La pregunta más frecuente al final de cualquier evaluación de modelos locales es la misma: ¿cuándo tiene sentido mantener todo en casa y cuándo conviene delegar a la nube? La respuesta correcta no es "depende" sin más contexto —es un framework de cuatro dimensiones que, evaluadas sistemáticamente, producen una decisión bien fundamentada en la mayoría de los casos. Esas cuatro dimensiones son privacidad, latencia, costo y capacidad del modelo. Y lo crítico es el orden: la privacidad es el filtro excluyente que se evalúa primero; los otros tres son criterios de optimización que se aplican solo cuando la privacidad no impone una restricción absoluta.

La **privacidad** determina si la elección existe en absoluto. Los datos de salud regulados por HIPAA, los datos financieros bajo PCI-DSS, la información personal identificable bajo GDPR, y la información legalmente privilegiada —contratos, litigios, comunicaciones de M&A— no pueden salir del perímetro de la organización sin controles específicos que las APIs de terceros no garantizan universalmente bajo las mismas condiciones que un sistema on-premise gestionado directamente. En estos casos el procesamiento local no es una opción de costo sino un requisito legal con consecuencias de compliance si no se cumple. Este criterio excluyente elimina la discusión para quizás el 30-40% de los casos de uso empresariales más valiosos: el asistente médico, el analizador de contratos, el sistema de auditoría financiera, o cualquier producto que toque datos de clientes con protección legal.

La **latencia** define qué arquitectura es físicamente viable. Para casos de uso edge —procesamiento en el dispositivo del usuario, aplicaciones que deben funcionar sin conectividad, entornos industriales con redes aisladas— el modelo local es el único camino posible: no hay API a la que llamar. Para aplicaciones de tiempo real donde el TTFT debe estar por debajo de 100 ms, solo los modelos pequeños (1B-3B) en GPU local de alta velocidad o en memoria unificada de Apple Silicon pueden cumplir ese umbral. Pero la latencia también puede favorecer la nube: una API de OpenAI o Anthropic accedida desde una red de baja latencia puede ser más rápida que un modelo 7B corriendo en CPU local o en hardware de

consumo sin GPU discreta. La latencia del modelo local depende críticamente del hardware disponible; la latencia de la nube depende de la conectividad y la carga del proveedor. Medirlas en el entorno real antes de decidir es la única forma de obtener números confiables.

El **costo** tiene una estructura no lineal que muchos análisis simplifican incorrectamente. La nube es más económica para volúmenes bajos o intermitentes: no hay capex, no hay costo de ingeniería para operar la infraestructura, se paga exactamente lo que se usa. A partir de cierto volumen de peticiones mensuales —que varía según el hardware local disponible y los precios del proveedor de nube, y que debe calcularse con los números reales del trimestre actual— el modelo local resulta más económico porque el costo fijo del hardware se amortiza entre más peticiones. Ese punto de inflexión es el break-even calculado en el capítulo anterior. Consultar los precios actuales en las páginas oficiales de cada proveedor antes de construir el modelo de costo: los precios de APIs de LLM han caído significativamente y seguirán haciéndolo, lo que desplaza el punto de break-even.

La **capacidad del modelo** es frecuentemente el factor decisivo cuando los otros tres no imponen una restricción. Para tareas que requieren razonamiento multi-paso complejo, análisis jurídico profundo, síntesis de información conflictiva o generación de código en frameworks poco frecuentes, los modelos de frontera en la nube (GPT-4o, Claude Opus, Gemini Ultra) producen resultados significativamente superiores a los modelos locales de 7B-13B disponibles en hardware de consumo. Para tareas más acotadas —clasificación de intención, resumen de documentos cortos, extracción de entidades en dominios bien definidos, respuestas FAQ— un modelo de 7B fine-tuneado en datos del dominio puede superar a un modelo generalista de frontera: tiene más conocimiento específico de la tarea y produce outputs más consistentes en el formato esperado.

Marco de decisión local vs nube

- **Datos clasificados o regulados:** procesamiento local es el default no negociable; HIPAA, PCI-DSS, GDPR y confidencialidad legal requieren controles de datos que las APIs de terceros no garantizan sin acuerdos específicos.
- **Tareas simples de alto volumen:** modelos locales de 3B-7B fine-tuneados en la tarea son más económicos y suficientemente capaces; clasificación, resumen corto, extracción de entidades en dominio conocido.
- **Tareas complejas de bajo volumen:** modelos de frontera en la nube producen calidad superior para razonamiento complejo; el costo por petición es alto pero el volumen bajo hace la factura total manejable.

- **Latencia crítica (<100 ms TTFT):** solo posible localmente con modelos 1B-3B en GPU dedicada o Apple Silicon; cualquier API pública añade un mínimo de 50-150 ms de latencia de red.
- **Disponibilidad y SLA:** las APIs de nube de proveedores tier-1 ofrecen SLA del 99,9% con failover entre regiones; la infraestructura local requiere inversión explícita en alta disponibilidad para alcanzar niveles equivalentes.

Nota del Arquitecto: El error más frecuente es hacer el análisis de forma binaria: "¿local o nube?" La respuesta correcta para la mayoría de los productos con más de dos tipos de casos de uso distintos es "ambas en capas distintas". La siguiente sección presenta el patrón router que implementa esta combinación de forma sistemática.

La decisión local/nube no es permanente: las condiciones que la informan cambian. Los precios de las APIs caen, el hardware local se vuelve más potente, los volúmenes del producto crecen. Revisar el análisis trimestralmente y estar dispuesto a migrar peticiones entre capas cuando el break-even cambia es la práctica de ingeniería que maximiza la eficiencia económica de la arquitectura a lo largo del tiempo.

Módulo 8 – Capítulo 10 – Sección 02

Patrón router: clasificar peticiones y dirigir las al modelo correcto según complejidad

El patrón router —también denominado LLM routing o model routing— es la pieza arquitectónica central de cualquier sistema híbrido: una capa de clasificación intermedia que analiza cada petición entrante y la dirige al modelo más apropiado según criterios de complejidad, sensibilidad de datos, costo proyectado o latencia aceptable, sin que el cliente que realiza la petición necesite saber qué modelo la procesará. Desde la perspectiva del cliente, existe una única API OpenAI-compatible; desde dentro del sistema, esa API es una función de routing que optimiza continuamente la distribución de carga entre modelos locales y de nube basándose en políticas configurables. El router es la abstracción que convierte la decisión estática "¿local o nube?" en una decisión dinámica y por-petición que se toma en 5-20 ms de overhead adicional.

La implementación más simple del router usa heurísticas determinísticas basadas en características estructurales de la petición: longitud del prompt (prompts cortos a modelo local, prompts mayores de 4K tokens al modelo de nube con mayor ventana de contexto), tipo de endpoint (embeddings y clasificaciones siempre local, generación de código complejo siempre nube), y presencia de PII detectada por Presidio (cualquier petición con PII activa el routing a modelo local dentro del perímetro, independientemente del resto de criterios). Las heurísticas son rápidas —no añaden latencia de modelo al path de la petición— pero tienen precisión limitada: no pueden diferenciar entre una pregunta corta trivial y una pregunta corta que requiere razonamiento profundo.

El siguiente nivel de sofisticación es un **clasificador de complejidad entrenado**: un modelo BERT o DeBERTa fine-tuneado en un dataset de pares (prompt, complejidad_etiquetada) que predice en una categoría de dificultad (simple / intermedia / compleja) con una latencia de inferencia de 10-30 ms en CPU. RouteLLM, el proyecto open source de LMSYS, va un paso más allá: entrena un clasificador basado en matrices de preferencia humana que predice si dos modelos producirán respuestas de calidad equivalente para una petición dada, y solo enruta a la nube cuando predice una diferencia de calidad

significativa. En sus benchmarks, RouteLLM puede mantener el 95% de la calidad de GPT-4o enrutando solo el 28% del tráfico a la nube —el 72% restante se sirve localmente con un modelo menor sin degradación perceptible por el usuario.

La **API unificada** es el requisito de diseño que hace que el router sea transparente: el router implementa exactamente el mismo contrato de interfaz que sus backends (el mismo schema de OpenAI Chat Completions API, los mismos campos en la respuesta, el mismo comportamiento de streaming). Los clientes no necesitan modificarse para soportar el routing; los headers de respuesta pueden opcionalmente incluir metadatos del modelo usado (`X-Model-Used: llama-3.1-8b-instruct`) para debugging y telemetría sin afectar la compatibilidad. La lógica de routing puede actualizarse sin modificar ningún cliente.

El router requiere su propio **ciclo de evaluación y mejora continua**. La política de routing (qué clasificador usar, qué umbrales de complejidad, qué modelos en cada capa) es un hiperparámetro del sistema que se optimiza basado en datos de producción: logs de feedback de usuarios (thumbs up/down), métricas de re-intentos (que señalan respuestas insatisfactorias), y el análisis de las peticiones que fueron roteadas a la nube y podrían haberse servido localmente con calidad equivalente. Un AB test que compara dos políticas de routing diferentes —umbral de complejidad más conservador vs más agresivo— con métricas de calidad y costo como variables de resultado es el método correcto para optimizar el punto de equilibrio.

Componentes de implementación del patrón router

- **Clasificador de complejidad:** desde heurísticas de longitud y palabras clave hasta modelos de clasificación fine-tuneados (BERT/DeBERTa); las heurísticas son más rápidas (o ms overhead de modelo) pero menos precisas; el clasificador entrenado añade 10-30 ms pero permite routing semántico.
- **Criterios multidimensionales:** complejidad de la tarea + presencia de PII + longitud de contexto + tipo de usuario (plan premium / estándar) + throughput actual del modelo local; el router combina estos criterios en una función de scoring configurable.
- **Fallback implícito:** si el modelo local produce error, timeout o respuesta de baja calidad, el router reencamina automáticamente a la nube sin intervención manual; el fallback es el safety net que hace que el sistema sea resiliente a degradaciones del modelo local.
- **API unificada OpenAI-compatible:** el cliente ve una única API; el router implementa el mismo schema que sus backends; los metadatos del modelo usado se exponen en

headers opcionales para debugging y telemetría.

- **AB testing de políticas de routing:** comparar umbrales distintos con métricas de calidad y costo como variables de resultado; la política de routing es un hiperparámetro que requiere tuning basado en datos reales de producción.

Nota del Arquitecto: El beneficio económico del router es no lineal respecto al volumen: con 1.000 peticiones al día el overhead de implementación del router no se amortiza; con 100.000 peticiones al día, rerouting del 70-80% a modelos locales puede reducir la factura de inferencia en nube en un 60-70% sin degradación perceptible de la calidad media. Calcular el volumen de break-even antes de invertir en el router.

La clave de la eficiencia económica de la arquitectura híbrida es que el router aprende con el tiempo: a medida que se acumulan datos de producción sobre qué peticiones se sirven bien localmente, la política de routing se afina y el porcentaje de tráfico local aumenta. El sistema se vuelve más eficiente con el uso, no a pesar del cambio sino como resultado directo de él.

Módulo 8 – Capítulo 10 – Sección 03

Fallback patterns: resiliencia cuando el modelo local falla o supera su capacidad

El fallback pattern en arquitecturas híbridas no es un plan de contingencia de última hora — es una característica de diseño de primera clase que define cómo el sistema responde cuando el modelo local no puede satisfacer una petición de forma satisfactoria. Un sistema que no tiene fallback definido falla ante el usuario cuando el modelo local tiene problemas; un sistema con fallback bien diseñado hace transparente esa degradación y entrega la mejor respuesta posible en todo momento. La diferencia entre los dos no es si los modelos locales pueden fallar —todos pueden— sino si el sistema está diseñado para manejar esos fallos con elegancia.

Los escenarios que deben activar el fallback cubren toda la gama de formas en que un modelo local puede no cumplir con lo requerido. El **fallo técnico** es el más obvio: error HTTP 5xx del servidor de inferencia, OOM (out of memory) por un prompt más largo de lo esperado que satura el KV cache, timeout por una respuesta que excede el tiempo máximo aceptable para el usuario. El **fallo de capacidad** es el más frecuente en sistemas en producción: el prompt tiene más tokens de los que el modelo local soporta en su `--ctx-size` (8.192 tokens para muchos modelos Ollama básicos vs 128K en modelos de frontera en la nube); la petición requiere conocimiento que el modelo local no tiene por ser demasiado especializado o demasiado reciente respecto a su fecha de corte de entrenamiento. El **fallo de calidad** es el más difícil de detectar: el modelo local produce una respuesta técnicamente válida (sin error, dentro del tiempo) pero de calidad insuficiente para el caso de uso. Detectar el fallo de calidad requiere un verificador post-procesamiento.

El **fallback basado en timeout** es la implementación más simple y la primera línea de defensa: se configura un tiempo máximo de espera para la primera respuesta del modelo local (por ejemplo, 5 segundos para el TTFT); si se supera, la petición se cancela y se reenvía a la nube. En Python, `asyncio.wait_for(local_inference_coroutine(), timeout=5.0)` implementa este comportamiento con manejo de `asyncio.TimeoutError` para activar el fallback. La elección del umbral de timeout es un trade-off: demasiado corto genera fallos

innecesarios en peticiones legítimas que simplemente tienen prompts largos; demasiado largo degrada la experiencia de usuario en los fallos reales.

El **circuit breaker** es el mecanismo que previene el thundering herd de fallbacks cuando el modelo local está degradado de forma persistente. Si el modelo local falla N veces en una ventana de tiempo W (por ejemplo, 10 errores en 60 segundos), el circuit breaker "abre" y enruta todo el tráfico directamente a la nube durante un tiempo de recuperación T (por ejemplo, 60 segundos), sin intentar el modelo local. Después del tiempo de recuperación, el circuit breaker entra en estado "half-open" y permite un porcentaje pequeño de tráfico al modelo local para verificar si se ha recuperado antes de cerrar de nuevo. La librería `tenacity` de Python o `pybreaker` implementan este patrón con configuración declarativa.

El **cascading fallback** introduce una jerarquía de modelos en lugar de la dicotomía simple local/nube: modelo local 7B → modelo local 13B (si existe) → API de nube de costo intermedio → modelo de frontera en la nube. Cada nivel se activa solo si el anterior falla o no supera el umbral de calidad. Este patrón es especialmente útil en organizaciones con múltiples modelos locales de distintas capacidades disponibles: el 7B maneja el 70% del tráfico, el 13B maneja el 20% de mayor complejidad que el 7B no puede resolver bien, y solo el 10% restante va a la nube. El overhead de latencia del cascading fallback (intentar un modelo, verificar, intentar el siguiente) se mitiga ejecutando los pasos de verificación de calidad en paralelo con la generación, o usando un clasificador pre-generación que predice directamente el nivel de modelo necesario.

Implementación de fallback patterns

- **Timeout-based fallback:** umbral estricto de TTFT para el modelo local; `asyncio.wait_for()` o timeouts HTTP en el proxy; cancelación limpia de la petición local antes de reenviar a la nube.
- **Circuit breaker:** N fallos en ventana W abren el circuito por T segundos; half-open para verificar recuperación; `pybreaker` o `tenacity` en Python; evita thundering herd cuando el modelo local está degradado.
- **Cascading fallback:** jerarquía 7B → 13B → cloud_económica → cloud_frontera; cada nivel activado por fallo o baja calidad del anterior; el clasificador pre-generación puede predecir el nivel necesario sin intentar los niveles inferiores.
- **Response quality gate:** clasificador binario ligero (DistilBERT o ONNX en CPU) que evalúa la respuesta del modelo local en menos de 50 ms; score bajo activa el fallback a la nube; entrenado en logs de feedback de producción (thumbs up/down).

- **Logging exhaustivo de fallbacks:** cada fallback registra el motivo (timeout / error / baja calidad / contexto largo), el modelo origen, el modelo destino y la latencia adicional; esta telemetría alimenta el ciclo de mejora del clasificador de calidad y del clasificador de routing.

Nota del Arquitecto: *El response quality gate es el componente más valioso y el más frecuentemente omitido. Sin él, el sistema enruta a la nube solo cuando hay fallos técnicos obvios, pero permite que respuestas de baja calidad del modelo local lleguen al usuario. Con el quality gate, el sistema puede detectar "el modelo respondió algo, pero es demasiado corto / está fuera de dominio / es incoherente" y activar el fallback silenciosamente. La inversión de implementar un verificador de calidad ligero se recupera en calidad media del sistema.*

La telemetría de fallbacks es un activo de gestión del conocimiento: el análisis de los logs revela qué tipos de peticiones fallan más frecuentemente en el modelo local y proporciona exactamente el conjunto de datos necesario para una siguiente ronda de fine-tuning que reduzca la tasa de fallback. El sistema de fallback se convierte así en el mecanismo de recopilación de datos para mejorar el modelo local, cerrando el ciclo de mejora continua de la arquitectura híbrida.

Módulo 8 – Capítulo 10 – Sección 04

Privacidad diferencial: procesar datos sensibles localmente, enviar datos genéricos a la nube

El patrón de privacidad diferencial en arquitecturas híbridas resuelve un problema real que muchos equipos enfrentan: quieren acceder a la capacidad superior de los modelos de frontera en la nube, pero sus datos contienen información que no puede salir del perímetro de la organización. La solución no es renunciar a los modelos de nube ni renunciar a los controles de privacidad —es dividir el pipeline de procesamiento según la sensibilidad de los datos, ejecutar las partes sensibles localmente, y enviar únicamente la información anonimizada a la nube para el procesamiento que requiere mayor capacidad. Este patrón no es un workaround: es una arquitectura deliberada que permite usar los mejores modelos del mundo con datos sensibles que de otra forma solo podrían procesarse con modelos locales de menor capacidad.

El flujo de trabajo del patrón tiene tres fases. En la **fase de anonimización local**, el texto original que contiene información sensible pasa por un pipeline de de-identificación que detecta y sustituye PII por tokens placeholder: `<PERSON_0>`, `<DATE_1>`, `<AMOUNT_2>`. Presidio de Microsoft es la librería de referencia para esta tarea en Python: soporta más de 20 tipos de entidades predefinidas (nombres, DNIs, emails, teléfonos, números de tarjetas de crédito, fechas, direcciones) y permite definir reconocedores personalizados para jerga específica del dominio mediante expresiones regulares o modelos NER propios. El texto anonimizado preserva la estructura gramatical y el contexto semántico del original; solo los valores concretos de PII se sustituyen, no las categorías de información.

En la **fase de procesamiento en nube**, el texto anonimizado se envía al modelo de frontera para el procesamiento que requiere su capacidad superior: síntesis de documentos complejos, análisis comparativo entre múltiples fuentes, generación de respuestas elaboradas que integran información de múltiples entidades. El modelo de nube procesa tokens placeholder como cualquier otro token; mientras el texto anonimizado preserve la coherencia semántica, la calidad del procesamiento no se degrada significativamente para la mayoría de

las tareas. La fase de nube nunca ve los valores reales de PII: solo ve su posición y categoría relativa en el texto.

En la **fase de restauración local**, la respuesta de la nube —que puede contener referencias a los mismos tokens placeholder— se postprocesa localmente para restaurar los valores reales donde corresponda. Para ello, la fase de anonimización mantiene un mapa en memoria asociado a la sesión del usuario: `{"<PERSON_0>": "María García", "<DATE_1>": "15 de enero de 2024"}`. La restauración recorre la respuesta de la nube, sustituye los tokens de vuelta por los valores reales, y entrega al usuario la respuesta final con los datos originales. Este mapa de sesión se mantiene únicamente en el servidor local dentro del perímetro y se descarta al terminar la sesión.

Un ejemplo concreto: en un asistente jurídico, el análisis de un contrato de 50 páginas con datos de las partes, fechas de obligaciones y montos económicos sigue este flujo. La fase local extrae con Presidio los nombres de las partes (`<PERSON_0>`, `<ORGANIZATION_1>`), las fechas clave (`<DATE_2>`), y los montos (`<AMOUNT_3>`). El texto anonimizado —que sigue siendo un contrato coherente y analizable— se envía al modelo de frontera en la nube para su análisis jurídico detallado, identificación de cláusulas problemáticas y comparación con cláusulas estándar del sector. La respuesta de análisis de la nube, que referencia `<ORGANIZATION_1>` en sus observaciones, se restaura localmente con el nombre real de la empresa antes de mostrarse al abogado. El modelo de frontera nunca supo el nombre de la empresa; el abogado recibe un análisis de alta calidad que usa el nombre real.

Implementación del patrón de privacidad diferencial

- **Presidio para de-identificación:** `presidio_analyzer` para detección de entidades, `presidio_anonymizer` para sustitución por tokens; soporta entidades predefinidas y recognizers custom en Python; disponible también como servicio HTTP independiente.
- **Reconocedores de dominio:** Presidio permite reconocedores NER fine-tuneados en la jerga específica del negocio (términos médicos, legales, financieros) más precisos que los reconocedores generales para jerga específica del dominio.
- **Tokenización reversible con mapa de sesión:** mantener el mapa `{token_placeholder → valor_real}` en el servidor local asociado a un session ID; los valores reales nunca salen del servidor local; el mapa se descarta al finalizar la sesión.
- **Evaluación de calidad con datos anonimizados:** la calidad del modelo de nube sobre texto con tokens placeholder puede diferir de su calidad sobre texto real; medir

explícitamente en un dataset de prueba representativo antes de adoptar el patrón en producción.

- **Auditoría de flujo de datos:** log local de qué datos (solo los textos anonimizados) se enviaron a la nube, qué proveedor los procesó y cuándo; componente de compliance que demuestra que los datos sensibles nunca salieron del perímetro.

Nota del Arquitecto: *El mayor riesgo de este patrón es la anonimización incompleta: si el modelo local de de-identificación falla en detectar una entidad de PII, esa PII sale del perímetro. La evaluación del recall del sistema de de-identificación sobre datos representativos del dominio es tan crítica como la evaluación del modelo de nube. Un sistema de de-identificación con recall del 95% deja pasar el 5% de las entidades de PII —en un sistema de atención médica eso puede ser inaceptable. El umbral de recall requerido debe definirse explícitamente como parte de los requisitos del sistema antes de adoptar el patrón.*

La privacidad diferencial como patrón de arquitectura no elimina todos los riesgos de privacidad —un adversario suficientemente sofisticado podría en teoría inferir información sensible a partir del contexto de texto anonimizado— pero los reduce a un nivel que, combinado con los controles contractuales con el proveedor de nube y la auditoría de flujo de datos, es aceptable para muchos entornos regulados. El marco de evaluación del riesgo residual debe validarse con el equipo legal y de compliance antes del despliegue en producción con datos realmente sensibles.

Módulo 8 – Capítulo 10 – Sección 05

Cost modeling: construir el modelo de costos de una arquitectura híbrida

El cost modeling de una arquitectura híbrida de LLMs es un ejercicio de contabilidad total que va mucho más allá de comparar el precio por millón de tokens de una API con el costo eléctrico de un servidor GPU. Los análisis superficiales que ignoran el costo de ingeniería para operar la infraestructura local, el costo de las regresiones de calidad cuando el modelo local produce respuestas subóptimas, o el costo del working capital inmovilizado en hardware comprado por adelantado producen decisiones erróneas que se manifiestan en sorpresas presupuestarias 6-12 meses después. El modelo de costo correcto captura todos los componentes en el mismo denominador: costo por petición o costo por 1M tokens procesados, incluyendo absolutamente todos los costos directos e indirectos de cada arquitectura.

El **costo de inferencia local** tiene dos componentes principales que deben sumarse. El costo de capital (capex) se amortiza mensualmente: precio total del hardware dividido por los meses de vida útil esperada (generalmente 36-48 meses para servidores GPU) dividido a su vez por las peticiones mensuales procesadas a la utilización promedio esperada. El costo operativo (opex) mensual incluye electricidad (una RTX 4090 bajo carga consume ~450W, multiplicado por las horas de operación al mes y el precio del kWh local), refrigeración adicional (aproximadamente el 20-30% del consumo eléctrico de los servidores), ancho de banda de red para descargas de modelos y actualizaciones, y espacio de rack o datacenter si aplica. El costo de ingeniería para operar la infraestructura es el ítem que más frecuentemente se omite: mantener un cluster de 2-4 GPUs en producción con actualizaciones de drivers, monitoring, gestión de fallos de hardware, rotación de modelos y guardia de incidentes requiere entre 0,2 y 0,5 FTE de ingeniería, que es el costo dominante en equipos con salarios de mercado.

El **costo de inferencia en nube** parece simple —precio publicado por input/output tokens— pero tiene costos ocultos que se acumulan. El costo de egress de datos cuando se envían documentos grandes como input (en AWS, el egress hacia internet tiene un costo por GB que puede ser relevante para pipelines de procesamiento de documentos de alto volumen). La

variabilidad de precios entre regiones del mismo proveedor (hasta 15-20% de diferencia entre us-east-1 y eu-west-1 para los mismos modelos). El costo de retokenización cuando se cambia de modelo: diferentes modelos usan tokenizadores distintos y el mismo texto puede producir 10-20% más o menos tokens según el modelo, lo que afecta directamente el costo. Y el overhead de latencia que en aplicaciones de alto valor puede traducirse en menor engagement del usuario —un costo de negocio difícil de cuantificar pero real.

El **costo de calidad diferencial** es el ítem más difícil de modelar pero potencialmente el más relevante. Si el modelo local produce un 5% más de respuestas que requieren re-intento o que generan contacto de soporte comparado con el modelo de nube, y cada uno de esos eventos tiene un costo (tiempo del usuario, llamada de soporte, churn), ese diferencial de calidad puede superar fácilmente el ahorro de infraestructura. Cuantificar el costo de calidad requiere datos de producción: tasa de re-intentos por modelo, tasa de escalación a soporte por modelo, y Net Promoter Score diferencial si hay suficiente volumen para ABtesting estadísticamente significativo.

El **análisis de escenarios de crecimiento** convierte el modelo de costos estático en una herramienta de decisión dinámica. Construir tres escenarios (bajo: 50% del volumen proyectado, base: 100%, alto: 200%) con el costo total mensual de la arquitectura híbrida, de solo local, y de solo nube en cada escenario revela cuándo conviene cada opción en la curva de crecimiento. Generalmente el resultado es: la nube pura es más económica a bajo volumen (el capex de hardware no se amortiza), la arquitectura híbrida es la más económica a volumen base y alto (el tráfico local amortiza el hardware mientras el tráfico de nube maneja los picos), y solo local es la más económica solo en el escenario de máxima utilización con hardware plenamente amortizado. Consultar los precios actuales de cada proveedor en sus páginas oficiales antes de construir el modelo —los precios de inferencia de LLM cambian con frecuencia.

Componentes del modelo de costos

- **Costo de inferencia local:** $(\text{capex_amortizado} + \text{opex_mensual} + \text{costo_ingeniería}) / \text{peticiones_mensuales}$; incluir ciclo de vida del hardware de 36-48 meses, electricidad + refrigeración, y 0,2-0,5 FTE de ingeniería de operaciones.
- **Costo de inferencia en nube:** precio publicado por input/output tokens consultado en las páginas oficiales del trimestre actual; más costo de egress de datos para inputs grandes, variación por región y overhead de retokenización al cambiar de modelo.

- **Costo de calidad diferencial:** medir tasa de re-intentos y escalación a soporte por modelo en producción; el diferencial de calidad tiene un costo de negocio real que puede dominar el análisis.
- **Análisis de escenarios:** tres escenarios de volumen (50%/100%/200%) con el costo total de cada arquitectura; el punto de break-even entre nube pura e híbrida cambia con el volumen y con los precios del proveedor.
- **Revisión trimestral:** el cost model es un documento vivo; actualizar cuando cambian los precios de proveedores, cuando se amortiza hardware, cuando el volumen de producción difiere significativamente del proyectado.

Nota del Arquitecto: *El error más frecuente en el cost modeling de LLMs es comparar el costo marginal de la nube (precio por token, cero capex) con el costo total de local (capex + opex + ingeniería). La comparación correcta es costo total de propiedad vs costo total de propiedad. Cuando se incluyen todos los componentes — incluyendo el costo de ingeniería para operar el hardware local— muchos proyectos descubren que el break-even está significativamente más lejos de lo que estimaban inicialmente, y que la nube pura es la opción correcta hasta volúmenes más altos de los esperados.*

El cost model correcto no lleva a una decisión única y permanente; lleva a una arquitectura adaptativa que migra peticiones entre capas conforme el volumen cambia y los precios evolucionan. La arquitectura híbrida es exactamente esa: no un diseño fijo sino un sistema que puede ajustar la proporción local/nube sin cambiar el código del cliente, simplemente actualizando la política de routing.

Módulo 8 – Capítulo 10 – Sección 06

Cierre: local y nube no son alternativas, son capas complementarias de una arquitectura

El recorrido a través del Módulo 8 ha sido, en esencia, el desmontaje de una dicotomía falsa. La pregunta con la que muchos ingenieros abordan los modelos locales —"¿debería usar local o nube?"— asume que la respuesta es una elección de arquitectura única y permanente. La práctica de los sistemas de IA más sofisticados del mercado en 2025 demuestra exactamente lo contrario: los productos de IA con mayor escala y mayor complejidad de requisitos son, invariablemente, arquitecturas de capas donde cada componente se ejecuta en el entorno más apropiado según sus requisitos específicos de privacidad, latencia, costo y capacidad del modelo. La dicotomía no existe; existe un espectro de configuraciones y el diseño correcto es el que combina las capas en la proporción óptima para cada producto.

El patrón de arquitectura emergente tiene forma de pirámide invertida según la frecuencia de uso: en la base, los casos de uso más frecuentes y más simples —embeddings, clasificación de intención, respuestas FAQ, resúmenes cortos— se sirven con modelos locales pequeños y rápidos que procesan el 70-80% del volumen total con una fracción del costo. En el nivel intermedio, los casos de uso de complejidad media —análisis de documentos, generación de respuestas de varios párrafos, razonamiento en dos pasos— se distribuyen entre modelos locales más grandes y APIs de nube de costo moderado según la sensibilidad de los datos y la latencia requerida. En el vértice superior, el 10-20% de los casos de uso más complejos o más críticos para el negocio —síntesis de información compleja, generación de contenido de alto valor, razonamiento multi-hop— se procesan con los modelos de frontera en la nube que garantizan la mayor calidad posible sin importar el costo por petición porque el volumen bajo hace la factura total manejable.

Esta arquitectura en capas convierte a los LLMs en servicios de primera clase dentro de la arquitectura de software del producto. No son herramientas externas a las que se llama de forma ad hoc; son componentes con SLOs de latencia y calidad definidos, con su propio ciclo de vida de versiones gestionado con las prácticas del Capítulo 9, con controles de acceso y mecanismos de observabilidad que producen las métricas que alimentan el ciclo de mejora

continua. El AI Engineer que diseña estos sistemas aplica exactamente los mismos principios que aplica a cualquier otro microservicio distribuido: disponibilidad, observabilidad, escalabilidad, degradación graceful y gobierno del ciclo de vida.

La complejidad que introduce esta arquitectura distribuida tiene contrapartida directa en capacidad, eficiencia y control. Capacidad porque combinar modelos locales y de nube da acceso a un rango de modelos imposible de alcanzar con cualquier capa única. Eficiencia porque el routing inteligente asigna cada petición al modelo más económico que puede resolverla con la calidad requerida. Y control porque los datos sensibles nunca salen del perímetro, los modelos fine-tuneados son artefactos propietarios con versionado y governance explícitos, y la arquitectura de fallback hace el sistema resiliente a la degradación de cualquier componente individual.

El siguiente módulo —Módulo 9— aborda la dimensión de seguridad que se extiende por toda esta arquitectura distribuida. Desplegar modelos locales, fine-tuned sobre datos propietarios, servidos a través de APIs públicas e integrados con sistemas empresariales introduce superficies de ataque específicas que no existen en el software convencional: prompt injection que explota la interfaz de lenguaje natural del modelo, exfiltración de datos a través de las respuestas del modelo, envenenamiento del dataset de fine-tuning que compromete el comportamiento del modelo en producción, y ataques de supply chain contra los pesos del modelo descargados de registros externos. El Módulo 9 examinará estas superficies de ataque y las defensas correspondientes: desde la evaluación de robustez ante prompt injection hasta la detección de comportamientos anómalos en inferencia, pasando por los controles de seguridad en el pipeline de fine-tuning y las políticas de acceso para sistemas multi-tenant. La arquitectura técnica que este módulo ha construido es el prerequisite; la seguridad que el siguiente módulo definirá es lo que la hace operable en producción con confianza.

Idea central

La arquitectura híbrida local/nube para LLMs no es un compromiso técnico sino el diseño correcto: cada capa tiene su espacio óptimo de uso, y combinarlas con routing inteligente produce sistemas más eficientes, más seguros y más resilientes que cualquier arquitectura de capa única.

"The network is the computer." — John Gage, Sun Microsystems, frase que en el contexto de los LLMs de 2025 adquiere una nueva dimensión: la arquitectura de inferencia distribuida —

local, nube y edge— es el sistema de IA, no el modelo individual. El diseño de cómo se conectan las capas, cómo el router toma decisiones en milisegundos, y cómo el fallback garantiza la resiliencia es exactamente tan importante como la calidad del modelo en cada nodo de esa red. El modelo es un componente del sistema; la arquitectura es el sistema.